

Phase 1 API Documentation

Ring Buffers

SPSC Ring Buffer

Header: `rings/spsc_ring.h`

Functions

```
spsc_ring_t* spsc_ring_create(size_t capacity);
```

Creates a new SPSC ring buffer with the specified capacity (rounded up to power of 2).

```
void spsc_ring_destroy(spsc_ring_t* ring);
```

Destroys the ring buffer and frees all resources.

```
ring_status_t spsc_ring_enqueue(spsc_ring_t* ring, void* element);
```

Enqueues an element (producer only). Returns `RING_SUCCESS` or `RING_ERROR_FULL`.

```
ring_status_t spsc_ring_dequeue(spsc_ring_t* ring, void** element);
```

Dequeues an element (consumer only). Returns `RING_SUCCESS` or `RING_ERROR_EMPTY`.

Example

```
// Create ring
spsc_ring_t* ring = spsc_ring_create(1024);

// Producer thread
void* data = malloc(100);
spsc_ring_enqueue(ring, data);

// Consumer thread
void* received = NULL;
spsc_ring_dequeue(ring, &received);

// Cleanup
spsc_ring_destroy(ring);
```

MPSC Ring Buffer

Header: `rings/mpsc_ring.h`

Similar API to SPSC, but supports multiple producers.

Fiber System

Fiber

Header: `fibers/fiber.h`

Functions

```
fiber_t* fiber_create(fiber_func_t entry, void* arg, size_t stack_size, uint32_t priority);
```

Creates a new fiber with the specified entry point, argument, stack size, and priority.

```
void fiber_destroy(fiber_t* fiber);
```

Destroys a fiber and frees resources.

```
void fiber_switch(fiber_t* from, fiber_t* to);
```

Low-level context switch between fibers.

Fiber Scheduler

Header: `fibers/fiber_scheduler.h`

Functions

```
fiber_scheduler_t* fiber_scheduler_create(uint32_t core_id);
```

Creates a per-core fiber scheduler.

```
uint64_t fiber_scheduler_spawn(fiber_scheduler_t* scheduler,
                               fiber_func_t entry,
                               void* arg,
                               size_t stack_size,
                               uint32_t priority);
```

Spawns a new fiber and adds it to the scheduler.

```
void fiber_scheduler_yield(fiber_scheduler_t* scheduler);
```

Yields current fiber to allow others to run.

```
void fiber_scheduler_run(fiber_scheduler_t* scheduler);
```

Runs the scheduler loop until all fibers complete.

Example

```
void my_fiber(void* arg) {
    printf("Hello from fiber!\n");
}

// Create scheduler
fiber_scheduler_t* scheduler = fiber_scheduler_create(0);

// Spawn fibers
fiber_scheduler_spawn(scheduler, my_fiber, NULL, 0, FIBER_PRIORITY_NORMAL);

// Run
fiber_scheduler_run(scheduler);

// Cleanup
fiber_scheduler_destroy(scheduler);
```

Shared Memory Arena

Header: arena/shared_arena.h

Functions

```
shared_arena_t* shared_arena_create(size_t size);
```

Creates a shared memory arena with the specified size.

```
void* shared_arena_alloc(shared_arena_t* arena, size_t size);
```

Allocates memory from the arena.

```
void* shared_arena_alloc_dma(shared_arena_t* arena, size_t size);
```

Allocates DMA-aligned memory (4KB alignment).

```
void shared_arena_free(shared_arena_t* arena, void* ptr, size_t size);
```

Frees memory back to the arena.

Example

```
// Create arena
shared_arena_t* arena = shared_arena_create(64 * 1024 * 1024);

// Allocate
void* buffer = shared_arena_alloc(arena, 4096);

// Use buffer...

// Free
shared_arena_free(arena, buffer, 4096);

// Cleanup
shared_arena_destroy(arena);
```

Zero-Copy IPC

Capability System

Header: `ipc/capability.h`

Functions

```
capability_t capability_create(void* base_address,
                             size_t size,
                             uint32_t permissions,
                             cap_trust_level_t trust_level,
                             uint64_t valid_from,
                             uint64_t valid_until);
```

Creates a capability for access control.

```
bool capability_validate(const capability_t* cap,
                        void* ptr,
                        size_t size,
                        uint32_t required_perms);
```

Validates a capability against access requirements.

Memory Descriptors

Header: `ipc/descriptor.h`

Functions

```
memory_descriptor_t descriptor_create(void* ptr,
                                     size_t length,
                                     capability_t capability,
                                     uint32_t flags,
                                     uint32_t owner_core);
```

Creates a memory descriptor for zero-copy transfer.

```
void descriptor_acquire(memory_descriptor_t* desc);
```

Acquires a reference to the descriptor.

```
bool descriptor_release(memory_descriptor_t* desc);
```

Releases a reference. Returns true if descriptor should be freed.

Zero-Copy Transfers

Header: `ipc/zero_copy.h`

Functions

```
zero_copy_status_t zero_copy_send(memory_descriptor_t* desc, uint32_t target_core);
```

Sends a descriptor to another core without copying data.

```
zero_copy_status_t zero_copy_map(const memory_descriptor_t* desc,
                                uint32_t required_perms,
                                void** ptr,
                                size_t* length);
```

Maps a descriptor for direct access.

Example

```
// Allocate buffer
void* buffer = shared_arena_alloc(arena, 4096);

// Create capability
capability_t cap = capability_create(buffer, 4096,
                                     CAP_PERM_READ | CAP_PERM_WRITE,
                                     CAP_TRUST_USER, 0, 0);

// Create descriptor
memory_descriptor_t desc = descriptor_create(buffer, 4096, cap, 0, 0);

// Send to another core (zero-copy!)
zero_copy_send(&desc, 1);

// Receiver maps and accesses
void* ptr;
size_t len;
zero_copy_map(&desc, CAP_PERM_READ, &ptr, &len);
// Use ptr directly - no copy!
```

Graph Calls

Header: `ipc/graph_call.h`

Functions

```
graph_call_port_t* graph_call_port_create(uint32_t core_id, size_t ring_capacity);
```

Creates a graph call port for syscall-free communication.

```
graph_call_status_t graph_call_submit(graph_call_port_t* port, graph_call_request_t* request);
```

Submits a graph call without syscall overhead.

```
graph_call_status_t graph_call_wait(graph_call_port_t* port,
                                     graph_call_request_t* request,
                                     uint64_t timeout_ns);
```

Waits for graph call completion.

Example

```
// Create port
graph_call_port_t* port = graph_call_port_create(0, 1024);

// Prepare request
graph_call_request_t request = {
    .type = GRAPH_CALL_MEMORY_ALLOC,
    .params.memory_alloc = {
        .size = 4096,
        .dma_capable = false
    }
};

// Submit (no syscall!)
graph_call_submit(port, &request);

// Wait for response
graph_call_wait(port, &request, 0);

// Use result
void* ptr = request.result.ptr;
```

Integration

Header: `integration/phase1_init.h`

Functions

```
phase1_status_t phase1_init(const phase1_config_t* config);
```

Initializes Phase 1 with the specified configuration.

```
phase1_status_t phase1_shutdown(void);
```

Shuts down Phase 1 and frees all resources.

```
phase1_config_t phase1_get_default_config(void);
```

Returns default configuration.

Example

```
// Get default config
phase1_config_t config = phase1_get_default_config();

// Customize
config.num_cores = 8;
config.arena_size = 128 * 1024 * 1024;

// Initialize
phase1_init(&config);

// Use Phase 1 components...

// Shutdown
phase1_shutdown();
```